Technical Deep Dive: API Architecture & Asynchronous Streaming

1. Introduction: The Restful Bridge

The communication layer between the React Single-Page Application (SPA) and the Python Django backend is a highly standardized, fully RESTful JSON API. Built atop the **Django REST Framework (DRF)**, this architecture enforces strict data schemas, protocol standards, and robust validation before any data touches the database.

However, standard REST architecture struggles with tasks that take longer than a few seconds or tasks that generate massive files. This document details the strict constraints of the API, the implementation of complex serializers, and the advanced patterns used to handle asynchronous AI polling and dynamic binary file streaming.

---

2. Strict RESTful Constraints & DRF Serializers

The API adheres strictly to REST (Representational State Transfer) architectural constraints, guaranteeing a predictable and scalable integration surface.

2.1 Statelessness

Every API request from the frontend must contain all the information the backend needs to process it. The Python server stores absolute zero session state about the client between requests. This allows the backend to horizontally scale across multiple Docker containers or Kubernetes pods seamlessly; a request to Server A requires no synchronization with Server B.

2.2 Resource-Oriented Routing

Endpoints are organized around logical database resources rather than imperative actions.

- **GET `/api/attendance/`:** Retrieves a list of records.
- **POST `/api/attendance/mark/`:** Creates a new record.
- **GET `/api/scheduler/subjects/`:** Retrieves a list of subjects.

2.3 Strict JSON Parsing (The Bouncers)

The Django REST Framework Serializers act as the system's strict bouncers. When the frontend sends a JSON payload (e.g., to create a new notice), the DRF Serializer intercepts the raw text stream.

1. **Type Validation:** It mathematically validates that the `priority` field is exactly an integer between 1 and 4.
2. **Constraint Validation:** It checks the database to ensure a notice with the exact same title doesn't already exist (enforcing unique constraints).
3. **Sanitization:** It scrubs the input of malicious HTML/JS to prevent Cross-Site Scripting (XSS) attacks.

If any of these checks fail, the Serializer halts execution entirely and automatically formats a structured `400 Bad Request` JSON response detailing exactly which field failed validation. The Python business logic never executes on dirty data.

---

3. Asynchronous Task Management (The Polling Pattern)

A standard HTTP request will generally time out if the server doesn't respond within 30 to 60 seconds. When an administrator triggers the AI Timetable Generation (which can take minutes of heavy math), a standard synchronous API call would fail catastrophically.

3.1 The 202 Accepted Trigger

To solve this, the `/api/scheduler/generate/` endpoint implements an asynchronous dispatch pattern.
When the `POST` request hits the endpoint, the Django view does not compute the timetable. It simply bundles the parameters and pushes them into the Django Q2 task queue.
The view instantly responds to the frontend with an HTTP status of `202 Accepted` and a JSON payload containing a unique identifier: `{ "task_id": "8f3a-4b2c-..." }`.

3.2 The Polling Endpoint

The React frontend then utilizes this `task_id` to continuously query a specialized, ultra-lightweight endpoint: `GET /api/scheduler/task-status/<task_id>/`.
This endpoint does no math. It executes a microsecond query against the Django Q2 status table. It returns the current state of the worker process: `QUEUED`, `RUNNING`, `SUCCESS`, or `FAILED`.
By pinging this endpoint every 2 seconds, the frontend can render a smooth, non-blocking progress bar. When the API finally returns `SUCCESS`, the frontend stops polling and reloads the primary data grids.

---

4. Dynamic Data Export & Binary Streaming

The API is not limited to returning plain JSON text. It functions as a dynamic file generation engine, constructing complex documents on the fly and streaming them directly into the user's browser.

4.1 CSV Streaming

When administrators require raw data for external reporting, they hit `/api/attendance/export/admin/csv/`.
Instead of querying the database, creating a massive temporary `.csv` file on the server's hard drive, and then sending a download link (which causes severe disk I/O bottlenecks and requires cleanup scripts), the API streams the data.
It utilizes Python's built-in `csv` module combined with Django's `StreamingHttpResponse`. The database iterates through records one by one, writes a line of CSV text to the open network socket, and flushes it immediately to the client. This uses practically zero server RAM, regardless of how massive the dataset is.

4.2 Dynamic PDF Generation (ReportLab)

For official student report cards, the API utilizes the `reportlab` library (`/api/attendance/export/student/pdf/`).
When the endpoint is hit, the Python backend instantiates a virtual PDF canvas entirely in RAM. It queries the database for the student's attendance metrics, draws the official university letterhead, constructs data tables with precise X/Y coordinates, and draws geometric pie charts.
Once the rendering is complete, the API attaches the raw binary byte-stream to the HTTP response with the header `Content-Type: application/pdf`, forcing the browser to instantly display the dynamically generated document without saving a single temporary file to the server.

4.3 Live Calendar Sync (iCalendar)

The API dynamically translates the complex relational timetable data (Rooms, Time Slots, Subjects) into the standardized RFC 5545 format (`.ics`).
By exposing the `/api/scheduler/ical/<user_id>/` endpoint, the system provides a live, continuously updating subscription feed. Students can paste this API URL directly into Apple Calendar or Google Calendar, and their phones will automatically parse the plain-text feed to render a fully native daily schedule.

---

5. Analytics KPI Feeds (Optimized Aggregations)

To render the complex, interactive charts on the dashboard, the frontend requires data structured in very specific ways.
Rather than dumping thousands of raw JSON records onto the frontend and forcing the client's CPU to calculate averages (which would destroy mobile phone battery life), the API features specialized analytical endpoints.

- **Pre-Formatted Trend Arrays:** The `/api/analytics/department-trends/` endpoint executes complex SQL `GROUP BY` and `AVG` operations at the database level. It then formats the result into multi-dimensional JSON arrays that perfectly map to the `recharts` React components. The frontend does zero math; it simply feeds the API response directly into the chart component, ensuring instant render times.